

untangle::executor

Generated by Doxygen 1.18.0

1 executor	1
2 Namespace Index	7
2.1 Namespace List	7
3 Class Index	9
3.1 Class List	9
4 File Index	11
4.1 File List	11
5 Namespace Documentation	13
5.1 untangle Namespace Reference	13
5.1.1 Detailed Description	13
6 Class Documentation	15
6.1 untangle::adaptive_mutex Class Reference	15
6.1.1 Detailed Description	15
6.2 untangle::executor< actionT > Class Template Reference	16
6.2.1 Detailed Description	16
6.2.2 Constructor & Destructor Documentation	17
6.2.2.1 executor()	17
6.2.2.2 ~executor()	17
6.2.3 Member Function Documentation	18
6.2.3.1 wait()	18
6.2.3.2 add_task()	18
6.2.3.3 pending()	19
6.2.3.4 start()	19
6.2.3.5 stop()	19
6.2.4 Member Data Documentation	20
6.2.4.1 on_task_error	20
7 File Documentation	21
7.1 executor.hpp	21
Index	27

Chapter 1

executor

c++ thread pool executor based on async execution

A header-only library: [executor.hpp](#) exposes a pool of workers, each one an [async](#) execution in continuous mode, fed by a shared queue of tasks.

requirements

C++23, and the async submodule:

```
git submodule update --init --recursive
```

async carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>
#include <functional>

int main() {
    untangle::executor<std::function<void(void)>> pool(4);
    pool.start();

    for (int i = 0; i < 100; ++i) {
        pool.add_task([i] { work(i); }, [] { /* finished */ });
    }

    return 0; // ~executor finishes what was added, then stops the workers
}
```

the action type

The pool is a template on the callable it runs, the way an execution is — and on the same parameter name, because every task queued is that one callable. One pool runs one signature, and the caller names it:

```
untangle::executor<std::function<int(void)>> pool(4); // each result goes to the task's callback
```

The action type does not have to be a `std::function`. What an execution requires of it is a nested `result_type`, which it reads rather than deduces, so any callable declaring one will do — and a pool built on such a callable never touches `std::function::result_type`, which the standard removed in C++20 and both `libc++` and `libstdc++` still supply by grace rather than contract.

```
struct my_task {
    using result_type = void;
    void operator()() const;
};

untangle::executor<my_task> pool(4);
```

tasks: work that reports back

Work enters the pool through one door, `add_task()`, and every task carries the callback it must notify. An action type may take arguments, and `add_task()` binds them the way its namesake `execution::add_task()` does — at the door, where the caller still holds them — with the callback after them:

```
untangle::executor<std::function<int(int)>> pool(4);
pool.start();

// the callback comes last, after the arguments the task is bound to
pool.add_task([](int n) { return n * 2; }, 21, [](int result) { /* result == 42 */ });
```

What waits for a worker is one callable carrying its own arguments and its callback, so nothing about a task changes between the queue and the worker that runs it. The arguments are copied at `add_task()` and handed to the task as the pool's own lvalues when it runs: a task is called later, possibly much later, and what the caller passed may be gone by then. A task taking `int&` therefore mutates the pool's copy, which the caller never sees.

A task that cannot be called with the arguments given is refused by a `static_assert` in `add_task()`, which names the mismatch before the errors from inside the binding follow it.

Work that returns `void` still reports that it finished, with a callback taking nothing — finished is the message and the result is optional:

```
untangle::executor<std::function<void(void)>> pool(4);
pool.add_task([] { /* ... */ }, [] { /* finished */ });
```

A task with nothing to bind passes the callback alone, whatever it returns:

```
untangle::executor<std::function<int(void)>> pool(4);
pool.add_task([] { return 42; }, [](int result) { /* result == 42 */ });
```

The callback is the last argument, and the signature cannot say so. A parameter pack cannot be followed by a deducible parameter, so it arrives inside the arguments and `untangle::bind_task()` splits it off. It must be callable with the task's result and return nothing — or callable with nothing at all, when the task returns `void`. A missing or unusable one is a compile error rather than silence.

Work nobody wants told about still passes a callback, one that ignores what it is given. There is no fire-and-forget door: a `void` task's callback costs one empty call, and a single door means a single queue, in which the order out is the order in.

```
constexpr auto ignore_result = [](auto&&...) {}; // takes a result, or nothing for a void task
pool.add_task([](int n) { return n * 2; }, 21, ignore_result);
```

A refused task does not notify. `add_task()` answers false before `start()`, after `stop()`, while a `wait()` is draining, once the destructor has begun, or if the worker it was offered to has stopped — and a refused

task is destroyed rather than queued. The answer is the whole report, so a caller waiting on the callback rather than on the answer would wait for ever. The same is true of work still queued when a stopped pool is destroyed: it never runs, so it never notifies, and the count on stderr is all there is.

Finished does not mean failed. A task that throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to `on_task_error`. And because a callback runs inside the same try as the task that owns it, `on_task_error` can fire for a task that in fact succeeded — the work was done and only the telling failed.

A callback runs on a worker's thread, inside that worker's drain, and several may be running at once on different workers. It is the same rule `on_task_error` carries, for the same reason: nothing may escape it.

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — `on_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `add_task()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

running, stopping, restarting

A pool is built stopped. The constructor makes the workers and wires them up; `start()` runs them. Until it is called the pool refuses work, because a task taken then would sit unrun and the destructor would wait for it. It is also the moment to assign `on_task_error`, before anything can throw.

`stop()` stops the workers, and stops the pool accepting. What a worker is already running it finishes; what is still queued stays queued. `stop()` does not wait for the workers to leave their threads — only the destructor does that, and that wait is what makes destroying the pool safe.

`start()` after `stop()` restarts it, workers and all, and the queue it was holding drains as they pick it up again. Either call is harmless twice over.

`wait()` is a barrier, not a shutdown. It returns once everything added has run and no worker is busy, and it leaves the pool exactly as it found it — workers still in their threads, nothing destroyed, accepting again. Work added after it runs as it did before, and calling it repeatedly is how you step through work in phases:

```
pool.add_task([ { first(); }, [] {}];
pool.wait();
pool.add_task([ { second(); }, [] {}];
pool.wait();
```

// first() has run, the pool is still alive
// and so has second()

It refuses new work while it waits, then restores accepting to whatever it was. A task that re-adds itself would otherwise keep the drain from ever ending, so `add_task()` answers false for the duration — and because the previous state is restored rather than assumed, `wait()` on a stopped pool does not leave it accepting.

So only work accepted before the wait began is what the wait covers. Work offered during it was never queued, and work added after it returns is simply the next batch, for the next `wait()`. This is a stop-the-world drain, not a transparent one: a producer on another thread sees `add_task()` answer false while a `wait()` is in progress, and has to offer the work again afterwards.

A pool that is not running is not waited for. Only a worker takes from the queue, so a stopped pool can never empty it, and a wait insisting otherwise would never return. `wait()` returns at once instead, leaving what is queued for the next `start()`.

lifetime and failure

The destructor finishes what was added. Its drain is `wait()`, so a task added before the pool goes out of scope has run by the time it does. What the destructor adds is the shutdown `wait()` deliberately leaves out, in the order that makes it safe: stop the workers, wait for every one of them to leave its thread, and only then destroy them.

The destructor waits for the pool, not for what the pool's work touched. That distinction is the whole reason `wait()` is public. The drain guarantees no worker is still running when the pool is destroyed, which is a statement about the pool's own members — a destructor body runs before them, so they are all still alive for it. It says nothing about the lifetime of what a task captured.

For a pool that is a local variable outliving what its work touches, the destructor is enough and nothing else is needed. For a pool held as a member of another class, it is already too late: `~executor()` runs during its owner's member destruction, so by then the owner's derived subobject is gone, and so is every member declared after the pool. A callback reaching into either of those runs against destroyed storage however faithfully the pool drains — and on the stack that is not even a fault a sanitizer can name, because the memory is still addressable.

So an owner whose callbacks reach back into itself calls `wait()` where the whole object graph is still standing — in its own destructor body, ahead of its members, or better still at a point the owner of its peers controls — and the destructor's drain goes back to being a backstop rather than the thing relied upon.

A pool destroyed while stopped is the exception: its queue can never empty, because only a running worker takes from it, so the destructor does not wait for it. Whatever is left is reported on `stderr` and dropped.

Both waits are unbounded, and both say so while they last. A task that never returns would otherwise leave the destructor silent for ever, so it reports instead — the first wait naming who is still inside a task, the second naming whose thread has not left. The first line below is the drain's, so a plain `wait()` prints it exactly as a destructor does; the second is the destructor's alone:

```
executor: still waiting to drain after 1s - 1 queued, 2 in a task: pool_1_worker_0, pool_1_worker_1
executor: still waiting to drain after 3s - 0 queued, 1 in a task: pool_1_worker_1
executor: still waiting to stop after 1s - 1 not left its thread: pool_1_worker_1
```

The interval starts at a second and doubles to a ceiling of thirty, so a stuck teardown says something almost at once without a long one becoming a flood. The two waits are kept apart because they answer different questions: a worker can be idle and still be in its thread.

Bounding either wait was never available. Returning early while a worker is still in a task is a use-after-free, and a bounded wait only moves the hang into `~execution()`, which spins on its own running state with no timeout. Workers are detached and `async::execution` has no cancellation, so a pool cannot outlive a task it cannot interrupt. What can be fixed is the silence.

`add_task()` says whether the task was taken. It returns false before `start()`, after `stop()`, while a `wait()` is draining, and once the destructor has begun. It returns false again if the worker it was offered to has stopped. A refused task is destroyed rather than queued, so a false answer means it will not run.

A task that is still running is refused too, and that is chosen rather than incidental. A pool draining is waiting for exactly that task, so it is tempting to let the work it spawns through — but then a task that re-adds itself could keep the drain from ever ending, and that holds for a `wait()` just as for a destructor. One rule instead: a pool that is not running takes no work, whoever is asking. `add_task()` answers false inside the task, so it can tell.

A task that throws does not take the worker with it, and what it threw is not lost. The worker catches it and carries on with the next task; `on_task_error` is where the throw goes:

```

untangle::executor<std::function<void(void)» pool(4);

pool.on_task_error = [](std::exception_ptr thrown) {
    try {
        std::rethrow_exception(thrown);
    } catch (const std::exception& e) {
        log(e.what());
    }
};

```

It carries a `std::exception_ptr` because a task may throw something that is not a `std::exception`, and that is the case most worth hearing about. Assign it before the first `add_task()` — a worker reads it — and expect it on a worker's thread, with more than one worker possibly inside it at once. Leave it unset and the pool warns on `stderr` instead, naming the worker.

The name says which pool too. Workers are named `<pool>_worker_<i>`, after the name the pool was given - `executor pool(2, "store_a")` has `store_a_worker_0` and `store_a_worker_1` - or, without one, `pool_<n>`, numbered across the process: two pools never share a worker name, so every warning - the failure reports above and the destructor's `waits_alike` - says where it came from.

A return value is not collected by the pool: a continuous worker keeps none. It goes to the task's own callback — see `tasks: work` that reports back above.

building the tests

```

cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build

```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access.

sanitizers

`EXECUTOR_SANITIZE` builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```

cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan

```

Accepted values are `address`, `thread`, `undefined`, or `empty`. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. `address` and `thread` instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

`tools/make_doc.sh` builds `doc/refman.pdf` from the header and this README. It needs `doxygen`, `tectonic` and `python3`, and it fails on any `doxygen` warning that is not an `obsolete-tag` notice, so an undocumented member or a broken `\ref` does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	13
--------------------------	---	--------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::adaptive_mutex	
A mutex that spins briefly before it blocks	15
untangle::executor< actionT >	
Runs tasks on a fixed pool of untangle::async::execution workers	16

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp	21
--	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class [adaptive_mutex](#)
A mutex that spins briefly before it blocks.
- class [executor](#)
Runs tasks on a fixed pool of `untangle::async::execution` workers.

Variables

- `std::atomic< std::size_t > unnamed_pools = {0}`
Numbers the pools that are not given a name, across every task type, so no two are alike.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

N workers in continuous mode behind a shared queue. Nothing overtakes what is already queued.

Chapter 6

Class Documentation

6.1 untangle::adaptive_mutex Class Reference

A mutex that spins briefly before it blocks.

```
#include <executor.hpp>
```

Public Member Functions

- void lock ()
Tries up to spin_limit times, then blocks until the lock is free.
- bool try_lock ()
Takes the lock if it is free, and never waits.
- void unlock ()
Releases the lock.

Static Public Attributes

- static constexpr int spin_limit = 100
How many times [lock\(\)](#) tries before it blocks.

6.1.1 Detailed Description

A mutex that spins briefly before it blocks.

The pool's lock is held for a few instructions at a time, by the submitter and by every worker after every batch. A `std::mutex` blocks in the kernel at once on macOS, so each meeting cost a pair of syscalls; spinning first ends most of them in user space. Measured with `bench/qt_pool_vs_this`.

The documentation for this class was generated from the following file:

- executor.hpp

6.2 `untangle::executor< actionT >` Class Template Reference

Runs tasks on a fixed pool of `untangle::async::execution` workers.

```
#include <executor.hpp>
```

Public Member Functions

- `executor` (`std::size_t worker_count`, `std::string name={}`)
Builds `worker_count` workers. They do not run until `start()`.
- `executor` (`const executor &`)=`delete`
Not copied or moved: every worker's callbacks hold this pool's this.
- `~executor` ()
Drains what was added, then stops the workers and destroys them.
- `void wait` ()
Drains everything added so far, then leaves the pool as it was found.
- `template<typename... Args>`
`bool add_task` (`actionT task`, `Args &&... args`)
Gives work that must report to a worker, which queues it.
- `std::size_t pending` () `const`
How many tasks are waiting in the workers' queues, not yet taken to run.
- `void start` ()
Starts every worker, and starts the pool accepting work.
- `void stop` ()
Stops every worker. What they are already running, they finish.

Public Attributes

- `std::function< void(std::exception_ptr)> on_task_error`
Called with whatever the work threw. Assigned by the caller.

6.2.1 Detailed Description

```
template<typename actionT>
class untangle::executor< actionT >
```

Runs tasks on a fixed pool of `untangle::async::execution` workers.

`add_task()` is the one door: it carries a callback and reports the result to it.

Each worker keeps its own queue. A task goes to a worker when it is submitted - the first idle one, else the one given the fewest since it last drained - and waits there for that worker.

Template Parameters

<code>actionT</code>	- The callable the pool runs. One pool runs one signature, arguments and result.
----------------------	--

6.2.2 Constructor & Destructor Documentation

6.2.2.1 executor()

```
template<typename actionT>
untangle::executor< actionT >::executor (
    std::size_t worker_count,
    std::string name = {}) [inline], [explicit]
```

Builds worker_count workers. They do not run until [start\(\)](#).

Parameters

worker_count	- How many workers to run. Must not be zero.
name	- What its workers are named after, <name>_worker_<i>, in every warning they print. Left empty, it is pool_<n>, numbered across the process, so two pools are never alike.

Exceptions

std::invalid_argument	- worker_count is zero: such a pool could never run its work.
-----------------------	---

6.2.2.2 ~executor()

```
template<typename actionT>
untangle::executor< actionT >::~~executor () [inline]
```

Drains what was added, then stops the workers and destroys them.

The drain is [wait\(\)](#); the shutdown after it stops the workers, waits for each to leave its thread, and destroys them only then. Both waits are unbounded and report on stderr meanwhile.

Remarks

Nothing is dropped: a worker runs what it was given before it leaves its thread, a pool stopped earlier included.

Attention

This waits for the pool, not for what its work touched: see [wait\(\)](#).

6.2.3 Member Function Documentation

6.2.3.1 wait()

```
template<typename actionT>
void untangle::executor< actionT >::wait () [inline]
```

Drains everything added so far, then leaves the pool as it was found.

Drained means no worker has anything queued or running, so every task accepted before this has run, and so has the callback of one - a callback runs inside the worker's drain, not after it.

A barrier, not a shutdown: the workers keep their threads and work added afterwards runs as before. Reusable, and one at a time - a second caller drains once the first has returned.

The wait is unbounded and reports on stderr while it lasts.

Attention

New work is refused meanwhile, then accepting is restored, so a task that re-adds itself cannot keep the drain from ending. Only work accepted before it began counts.

Remarks

A stopped pool is waited for too: its workers run what they were given before they leave, so the wait ends.

Attention

It waits for the work, not for what the work touched. Held as a member, [~executor\(\)](#) runs once the owner's derived parts and later members are gone: call this before that.

6.2.3.2 add_task()

```
template<typename actionT>
template<typename... Args>
bool untangle::executor< actionT >::add_task (
    actionT task,
    Args &&... args) [inline]
```

Gives work that must report to a worker, which queues it.

The first idle worker takes it, the warmest one; otherwise the worker given the fewest since it last drained, and the task waits behind that worker's queue - in the order it arrived there. Once the task has run, its callback is invoked with the result, on the worker's thread.

Attention

The last argument is the callback, which the signature cannot say: it arrives in args and [untangle::bind_task\(\)](#) splits it off. A missing or unusable one is a `static_assert`, and so is a task that cannot be called with the arguments before it.

A task that throws is not notified - what it threw goes to [on_task_error](#) - and a refused one does not notify either, so a caller waiting on the callback alone waits for ever.

An empty callback or action is refused here too, silently: the answer is the report. Only a run time check can see it, and a task that can neither run nor notify is turned away before any worker is asked.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.
------	--

Parameters

task	- The task to run.
args	- What to bind, followed by the callback. Copied by the worker's queue, so a task taking a reference gets the pool's copy.

Returns

true - accepted, and the callback will be notified once it has run.

false - refused by a pool not started, stopped, draining or being destroyed, by a stopped worker, or because the task could never do its job. It will not run or notify.

6.2.3.3 pending()

```
template<typename actionT>
std::size_t untangle::executor< actionT >::pending () const [inline]
```

How many tasks are waiting in the workers' queues, not yet taken to run.

Remarks

Advisory: true the moment it is read.

6.2.3.4 start()

```
template<typename actionT>
void untangle::executor< actionT >::start () [inline]
```

Starts every worker, and starts the pool accepting work.

A pool refuses work until this is called, and again after [stop\(\)](#) until it is called anew, which revives the workers too. Harmless twice over. [wait\(\)](#) needs nothing from here.

6.2.3.5 stop()

```
template<typename actionT>
void untangle::executor< actionT >::stop () [inline]
```

Stops every worker. What they are already running, they finish.

What the workers were given they still run, then leave; anything added afterwards is refused, a running task included. [start\(\)](#) revives the workers; harmless twice over.

Attention

It does not wait for the workers to leave their threads - only the destructor does, and that wait is what makes destroying the pool safe.

6.2.4 Member Data Documentation

6.2.4.1 `on_task_error`

```
template<typename actionT>  
std::function<void(std::exception_ptr)> untangle::executor< actionT >::on_task_error
```

Called with whatever the work threw. Assigned by the caller.

The only report of failed work, since [add_task\(\)](#) answered long before it ran. Left unset, the pool warns on stderr. Rethrow the `exception_ptr` to read it.

Remarks

A task's failure arrives here, and so does its callback's - so this can fire for a task that succeeded, when only the telling failed.

Attention

It runs on a worker's thread, several at once. Assign it before the first [add_task\(\)](#), and let nothing escape it or a callback.

The documentation for this class was generated from the following file:

- `executor.hpp`

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00008 #ifndef UNTANGLE_EXECUTOR_EXECUTOR_HPP
00009 #define UNTANGLE_EXECUTOR_EXECUTOR_HPP
00010
00011 #include <algorithm>
00012 #include <async.hpp>
00013 #include <atomic>
00014 #include <chrono>
00015 #include <condition_variable>
00016 #include <cstdint>
00017 #include <exception>
00018 #include <functional>
00019 #include <memory>
00020 #include <mutex>
00021 #include <print>
00022 #include <stdexcept>
00023 #include <string>
00024 #include <thread>
00025 #include <tuple>
00026 #include <type_traits>
00027 #include <utility>
00028 #include <vector>
00029
00030 #ifdef _MSC_VER
00031 #include <intrin.h>
00032 #endif
00033
00034 namespace untangle {
00035
00044 class adaptive_mutex {
00045 public:
00047     static constexpr int spin_limit = 100;
00048
00050     void lock() {
00051         for (int attempt = 0; attempt < spin_limit; ++attempt) {
00052             if (mutex_.try_lock()) {
00053                 return;
00054             }
00055             cpu_pause();
00056         }
00057         mutex_.lock();
00058     }
00059
00061     bool try_lock() { return mutex_.try_lock(); }
00062
00064     void unlock() { mutex_.unlock(); }
00065
00066 private:
00068     static void cpu_pause() noexcept {
00069         #if defined(_MSC_VER) && (defined(_M_X64) || defined(_M_IX86))
00070             _mm_pause();
00071         #elif defined(_MSC_VER) && defined(_M_ARM64)
00072             __yield();
00073         #elif defined(__x86_64__) || defined(__i386__)
00074             __builtin_ia32_pause();
00075         #elif defined(__aarch64__) || defined(__arm__)
```

```

00076     __asm__ __volatile__ ("yield");
00077 #endif
00078 }
00079
00080 std::mutex mutex_;
00081 };
00082
00083 inline std::atomic<std::size_t> unnamed_pools = {0};
00084
00085 template <typename actionT>
00086 class executor {
00087 public:
00088     explicit executor(std::size_t worker_count, std::string name = {}) {
00089         if (worker_count == 0) {
00090             throw std::invalid_argument("executor: worker_count must not be zero");
00091         }
00092
00093         if (name.empty()) {
00094             name = "pool_" + std::to_string(unnamed_pools.fetch_add(1) + 1);
00095         }
00096
00097         workers_.reserve(worker_count);
00098         given_ = std::make_unique<std::atomic<std::size_t>>>(worker_count);
00099
00100         for (std::size_t index = 0; index < worker_count; ++index) {
00101             auto next = worker_execution::create_instance(name + "_worker_" + std::to_string(index));
00102
00103             // Tells the pool this worker has run everything it was given.
00104             next->on_finished = [this, index] { worker_drained(index); };
00105
00106             // Carries what a task throws to the caller. Assigned here because a worker thread reads it.
00107             next->on_error = [this, worker = next->name]() { std::exception_ptr thrown) {
00108                 report_task_error(worker, thrown);
00109             };
00110
00111             // Adds the worker to the poll the destructor waits on.
00112             poll_.add(*next);
00113
00114             workers_.push_back(std::move(next));
00115         }
00116     }
00117
00118     executor(const executor&) = delete;
00119     executor& operator=(const executor&) = delete;
00120     executor(executor&&) = delete;
00121     executor& operator=(executor&&) = delete;
00122
00123     ~executor() {
00124         // Nothing is stopped before this, or the queue could never empty. wait() leaves the pool
00125         // accepting, so stop() follows at once: that window is the only one a shutdown would take.
00126         wait();
00127         stop();
00128
00129         // Waits for every worker to leave its thread, which is what makes destroying them safe. A
00130         // different question from the wait above: a worker can be idle and still be in its thread.
00131         // A worker leaves within microseconds of stop(), so the first tick is short and doubles up to
00132         // the ceiling: a fixed tick made every destruction sleep it once.
00133         auto interval = std::chrono::milliseconds(report_first_ms);
00134         auto waited = std::chrono::microseconds(0);
00135         auto tick = std::chrono::microseconds(poll_first_us);
00136
00137         while (poll_.is_running()) { // time of check
00138             std::this_thread::sleep_for(tick);
00139             waited += tick;
00140             tick = std::min(tick * 2, std::chrono::microseconds(poll_max_ms * 1000));
00141
00142             if (waited >= interval) {
00143                 std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00144                     std::chrono::duration_cast<std::chrono::seconds>(waited).count(),
00145                     count_workers(&worker_execution::is_running),
00146                     name_workers(&worker_execution::is_running));
00147
00148                 interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00149             }
00150         }
00151
00152         // time of use: no worker thread is left to reach workers_.
00153         workers_.clear();
00154     }
00155
00156     void wait() {
00157         // One drain at a time: two threads between the exchange below and the restore at the end would
00158         // read each other's cleared flag, and the last one out could leave the pool refusing work.
00159         std::lock_guard<std::mutex> drain(wait_mutex_);
00160
00161         // Put back exactly as found below: a wait must not leave a stopped pool accepting.
00162         const bool was_running = running_.exchange(false);

```

```

00214
00215 {
00216     std::unique_lock<adaptive_mutex> lock(mutex_);
00217
00218     // Counted under the lock, before the first check: a worker that drains after this sees it and
00219     // notifies, and one that drained before it is seen by that first check.
00220     ++waiting_;
00221
00222     // Waits for every worker to go idle.
00223     auto interval = std::chrono::milliseconds(report_first_ms);
00224     auto waited = std::chrono::milliseconds(0);
00225
00226     while (!finished_cv_.wait_for(lock, interval, [this] { return nothing_running(); })) {
00227         waited += interval;
00228
00229         std::println(
00230             stderr, "executor: still waiting to drain after {}s - {} queued, {} in a task{}",
00231             waited.count() / 1000, total_pending(), count_workers(&worker_execution::is_busy),
00232             name_workers(&worker_execution::is_busy));
00233
00234         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00235     }
00236
00237     --waiting_;
00238 }
00239
00240 running_ = was_running;
00241 }
00242
00270 template <typename... Args>
00271 bool add_task(actionT task, Args&&... args) {
00272     // The call the pool will make, which is not quite the one written at the call site: what
00273     // reaches the task is the copies the worker binds, so that is the call that has to compile.
00274     // Without this the mismatch surfaces deep inside bind_task()'s std::apply.
00275     static_assert(invocable_with_bound<Args...>(),
00276         "executor: the task cannot be called with the arguments given to add_task() "
00277         "before its callback");
00278
00279     if (!running_ || task_is_empty(task, args...)) {
00280         return false;
00281     }
00282
00283     return give_to_worker(pick_worker(), std::move(task), std::forward<Args>(args)...);
00284 }
00285
00291 std::size_t pending() const { return total_pending(); }
00292
00299 void start() {
00300     // The workers first, so anything accepted after this is handed to one that is running.
00301     for (auto& one : workers_) {
00302         one->start();
00303     }
00304
00305     running_ = true;
00306 }
00307
00318 void stop() {
00319     // First, so that nothing is accepted for workers that are about to stop. A task already on a
00320     // worker still runs; one arriving from here is refused.
00321     running_ = false;
00322
00323     // No lock: a worker draining after stop() runs on_finished, which may take mutex_.
00324     for (auto& one : workers_) {
00325         one->stop();
00326     }
00327 }
00328
00341 std::function<void(std::exception_ptr)> on_task_error;
00342
00343 private:
00350 void report_task_error(const std::string& worker, std::exception_ptr thrown) {
00351     if (on_task_error) {
00352         on_task_error(thrown);
00353         return;
00354     }
00355
00356     try {
00357         std::rethrow_exception(thrown);
00358     } catch (const std::exception& e) {
00359         std::println(stderr, "warning: executor task on '{}' threw: {}", worker, e.what());
00360     } catch (...) {
00361         std::println(stderr, "warning: executor task on '{}' threw an unknown type", worker);
00362     }
00363 }
00364
00374 template <typename... Args>
00375 static constexpr bool invocable_with_bound() {

```

```

00376     if constexpr (sizeof...(Args) == 0) {
00377         return true;
00378     } else {
00379         return invocable_with<std::tuple<std::decay_t<Args>...>>(
00380             std::make_index_sequence<sizeof...(Args) - 1>{});
00381     }
00382 }
00383
00385 template <typename tupleT, std::size_t... index>
00386 static constexpr bool invocable_with(std::index_sequence<index...>) {
00387     return std::is_invocable_v<actionT, std::tuple_element_t<index, tupleT>&...>;
00388 }
00389
00390 using worker_execution = untangle::async::execution<actionT>;
00391
00397 std::size_t pick_worker() const {
00398     std::size_t best = 0;
00399     std::size_t best_count = given_[0].load(std::memory_order_relaxed);
00400
00401     for (std::size_t index = 0; index < workers_.size(); ++index) {
00402         const std::size_t count = given_[index].load(std::memory_order_relaxed);
00403         if (count == 0) {
00404             return index;
00405         }
00406         if (count < best_count) {
00407             best = index;
00408             best_count = count;
00409         }
00410     }
00411
00412     return best;
00413 }
00414
00423 template <typename... Args>
00424 bool give_to_worker(std::size_t index, actionT task, Args&&... args) {
00425     // No lock: the counts only steer pick_worker(), so a stale one costs balance, not correctness.
00426     given_[index].fetch_add(1, std::memory_order_relaxed);
00427
00428     if (workers_[index]->add_task(std::move(task), std::forward<Args>(args)...)) {
00429         return true;
00430     }
00431
00432     // Only a stopped worker refuses: add_task() turned an empty task away before. Not counted, or
00433     // an idle worker would look busy until it is next given work and drains.
00434     forget_given(index);
00435     return false;
00436 }
00437
00443 template <typename... Args>
00444 static bool task_is_empty(const actionT& task, const Args&... args) {
00445     if constexpr (testable_for_emptiness<actionT>) {
00446         if (!task) {
00447             return true;
00448         }
00449     }
00450
00451     if constexpr (sizeof...(Args) > 0) {
00452         const auto& callback = std::get<sizeof...(Args) - 1>(std::forward_as_tuple(args...));
00453         if constexpr (testable_for_emptiness<std::decay_t<decltype(callback)>>) {
00454             if (!callback) {
00455                 return true;
00456             }
00457         }
00458     }
00459
00460     return false;
00461 }
00462
00466 void forget_given(std::size_t index) {
00467     std::size_t count = given_[index].load(std::memory_order_relaxed);
00468     while (count > 0 &&
00469         !given_[index].compare_exchange_weak(count, count - 1, std::memory_order_relaxed)) {
00470     }
00471 }
00472
00476 void worker_drained(std::size_t index) {
00477     given_[index].store(0, std::memory_order_relaxed);
00478
00479     // The lock only for a wait(), which counts itself under it before its first check.
00480     if (waiting_.load() > 0) {
00481         std::lock_guard<adaptive_mutex> lock(mutex_);
00482         finished_cv_.notify_all();
00483     }
00484 }
00485
00487 std::size_t total_pending() const {
00488     std::size_t total = 0;

```

```

00489     for (const auto& one : workers_) {
00490         total += one->pending();
00491     }
00492     return total;
00493 }
00494
00500 bool nothing_running() const {
00501     for (const auto& one : workers_) {
00502         if (one->is_busy()) {
00503             return false;
00504         }
00505     }
00506     return true;
00507 }
00508
00509 std::size_t count_workers(bool (worker_execution::*state)() const) const {
00510     std::size_t count = 0;
00511
00512     for (const auto& one : workers_) {
00513         if ((one.get()->*state)()) {
00514             ++count;
00515         }
00516     }
00517     return count;
00518 }
00519
00520 std::string name_workers(bool (worker_execution::*state)() const) const {
00521     std::string names;
00522
00523     for (const auto& one : workers_) {
00524         if ((one.get()->*state)()) {
00525             names += names.empty() ? " : " : ", ";
00526             names += one->name;
00527         }
00528     }
00529     return names;
00530 }
00531
00532 static constexpr int poll_first_us = 100;
00533 static constexpr int poll_max_ms = 50;
00534
00535 static constexpr int report_first_ms = 1000;
00536 static constexpr int report_max_ms = 30000;
00537
00538 mutable adaptive_mutex mutex_;
00539 std::condition_variable_any finished_cv_;
00540
00541 std::mutex wait_mutex_;
00542
00543 async::execution_poll poll_;
00544
00545 std::vector<std::shared_ptr<worker_execution>> workers_;
00546
00547 std::unique_ptr<std::atomic<std::size_t>> given_;
00548
00549 std::atomic<std::size_t> waiting_ = {0};
00550 std::atomic_bool running_ = {false};
00551 };
00552
00553 } // namespace untangle
00554
00555 #endif // UNTANGLE_EXECUTOR_EXECUTOR_HPP

```


Index

- ~executor
 - untangle::executor< actionT >, [17](#)
- add_task
 - untangle::executor< actionT >, [18](#)
- executor, [1](#)
 - untangle::executor< actionT >, [17](#)
- on_task_error
 - untangle::executor< actionT >, [20](#)
- pending
 - untangle::executor< actionT >, [19](#)
- start
 - untangle::executor< actionT >, [19](#)
- stop
 - untangle::executor< actionT >, [19](#)
- untangle, [13](#)
- untangle::adaptive_mutex, [15](#)
- untangle::executor< actionT >, [16](#)
 - ~executor, [17](#)
 - add_task, [18](#)
 - executor, [17](#)
 - on_task_error, [20](#)
 - pending, [19](#)
 - start, [19](#)
 - stop, [19](#)
 - wait, [18](#)
- wait
 - untangle::executor< actionT >, [18](#)